

C++ Standard Library Issues to be moved in Belfast

Doc. no. P1917R0

Date: Revised 2019-10-07 at 02:37:36 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Ready Issues

[3070^{\(i\)}](#). `path::lexically_relative` causes surprising results if a filename can also be a *root-name*

Section: 29.11.7.4.11 [[fs.path.gen](#)] Status: [Ready](#) Submitter: Billy O'Neal III Opened: 2018-02-23 Last modified: 2019-07-22

Priority: 2

View all other [issues](#) in [[fs.path.gen](#)].

View all issues with [Ready](#) status.

Discussion:

`path::lexically_relative` constructs the resulting path with operator/=. If any of the filename elements from **this* are themselves acceptable *root-names*, operator/= will destroy any previous value, and take that `root_name()`. For example:

```
path("/a:/b:").lexically_relative("/a:/c:")
```

On a POSIX implementation, this would return `path("../b:")`, but on a Windows implementation, the `"b:"` element is interpreted as a *root-name*, and clobbers the entire result path, giving `path("b:")`. We should detect this problematic condition and fail (by returning `path()`).

[2019-01-20 Reflector prioritization]

Set Priority to 2

[2019 Cologne Wednesday night]

Status to Ready

Proposed resolution:

This wording is relative to [N4727](#).

1. Change 29.11.7.4.11 [[fs.path.gen](#)] as indicated:

```
path lexically_relative(const path& base) const;
```

-3- [...]

-4- *Effects:* If `root_name() != base.root_name()` is true or `is_absolute() != base.is_absolute()` is true or `!has_root_directory() && base.has_root_directory()` is true or if any *filename in relative path()* or *base.relative path()* can be interpreted as a *root-name*, returns `path()`. [Note: On a POSIX implementation, no filename in a relative-path is acceptable as a root-name — end note] Determines the first mismatched element of **this* and *base* as if by:

```
auto [a, b] = mismatch(begin(), end(), base.begin(), base.end());
```

Then,

(4.1) — if `a == end()` and `b == base.end()`, returns `path(".");` otherwise

(4.2) — let n be the number of *filename* elements in `[b, base.end())` that are not dot or dot-dot minus the number that are dot-dot. If $n < 0$, returns `path()`; otherwise

(4.3) — returns an object of class `path` that is default-constructed, followed by

(4.3.1) — application of operator `/=` (`path("../")`) n times, and then

(4.3.2) — application of operator `/=` for each element in `[a, end())`.

3103⁽ⁱ⁾. Errors in taking subview of span should be ill-formed where possible

Section: 22.7.3.3 [\[span.sub\]](#) **Status:** [Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2018-04-13 **Last modified:** 2019-07-22

Priority: 3

View all issues with [Ready](#) status.

Discussion:

Currently all out-of-bound/inputs errors in the functions taking an subview of span lead to undefined behavior, even in the situation when they could be detected at compile time. This is inconsistent with the behavior of the span constructors, which make similar constructs ill-formed.

Furthermore, with the current specification of the `subspan` function, the following invocation:

```
span<T, N> s;    // N > 0
s.subspan<0>(); // with 0 > 0
```

is ill-formed when $0 > N + 1$, as the return of the function is `span<T, K>` with $K < -1$. However in case when $0 == N + 1$, runtime sized span is returned (`span<T, -1>`) instead and the behavior of the function is undefined.

Firstly, for either run time sized ($N == \text{dynamic_extent}$) and fixed sized ($N > 0$) object `s` of type `span<T, N>`, the following constructs should be ill-formed, instead of having undefined behavior:

1. `s.first<C>()` with $C < 0$
2. `s.last<C>()` with $C < 0$
3. `s.subspan<0, E>` with $0 < 0$ or $E < 0$ and $E != \text{dynamic_extent}$.

This would follow span specification, that make instantiation of `span<T, N>` ill-formed for $N < 0$ and $N != \text{dynamic_extent}$.

In addition the following constructs should be made ill-formed for fixed size span `s` of type `span<T, N>` (with $N > 0$):

1. `s.first<C>()` with $C > N$
2. `s.last<C>()` with $C > N$
3. `s.subspan<0, dynamic_extent>()` with $0 > N$
4. `s.subspan<0, C>()` with $0 + C > N$

This will match the span constructor that made construction of fixed size `span<T, N>` from fixed size span of different size ill-formed.

[2018-04-24 Priority set to 3 after discussion on the reflector.]

[2018-11 San Diego Thursday night issue processing]

Tomasz to provide updated wording.

Previous resolution: [SUPERSEDED]

This wording is relative to [N4741](#).

1. Edit 22.7.3.3 [\[span.sub\]](#) as indicated:

```
template<ptrdiff_t Count> constexpr span<element_type, Count> first() const;
```

-?- Remarks: If Count < 0 || (Extent != dynamic_extent && Count > Extent), the program is ill-formed.

-1- Requires: ~~0 <= Count &&~~ Count <= size().

-2- Effects: Equivalent to: return {data(), Count};

```
template<ptrdiff_t Count> constexpr span<element_type, Count> last() const;
```

-?- Remarks: If Count < 0 || (Extent != dynamic_extent && Count > Extent), the program is ill-formed.

-3- Requires: ~~0 <= Count &&~~ Count <= size().

-4- Effects: Equivalent to: return {data() + (size() - Count), Count};

```
template<ptrdiff_t Offset, ptrdiff_t Count = dynamic_extent>
constexpr span<element_type, see below> subspan() const;
```

-?- Remarks: The program is ill-formed if:

- Offset < 0 || (Count < 0 && Count != dynamic_extent), or
- Extent != dynamic_extent && (Offset > Extent || (Count != dynamic_extent && Offset + Count > Extent)).

-5- Requires: ~~{0 <= Offset && Offset <= size()} && (Count == dynamic_extent || Count >= 0 && Offset + Count <= size())~~.

-6- Effects: Equivalent to: return span<ElementType, see below>(data() + Offset, Count != dynamic_extent ? Count : size() - Offset);

-7- Remarks: The second template argument of the returned span type is:

```
Count != dynamic_extent ? Count
                        : (Extent != dynamic_extent ? Extent - Offset
                                                         : dynamic_extent)
```

[2018-11-09; Tomasz provides updated wording]

I have decided to replace all *Requires:* elements in the section 22.7.3.3 [\[span.sub\]](#) to preserve consistency.

Previous resolution: [SUPERSEDED]

This wording is relative to [N4778](#).

1. Edit 22.7.3.3 [\[span.sub\]](#) as indicated:

```
template<ptrdiff_t Count> constexpr span<element_type, Count> first() const;
```

-?- Mandates: Count >= 0 && (Extent == dynamic_extent || Count <= Extent).

-1- ~~Requires~~Expects: ~~0 <= Count &&~~ Count <= size().

-2- Effects: Equivalent to: return {data(), Count};

```
template<ptrdiff_t Count> constexpr span<element_type, Count> last() const;
```

-?- Mandates: Count >= 0 && (Extent == dynamic_extent || Count <= Extent).

-3- ~~Requires~~Expects: ~~0 <= Count &&~~ Count <= size().

```

-4- Effects: Equivalent to: return {data() + (size() - Count),
Count};

template<ptrdiff_t Offset, ptrdiff_t Count = dynamic_extent>
constexpr span<element_type, see below> subspan() const;

-?- Mandates: Offset >= 0 && (Count >= 0 || Count ==
dynamic_extent) && (Extent == dynamic_extent || (Offset <=
Extent && (Count == dynamic_extent || Offset + Count <=
Extent))).

-5- RequiresExpects: {0 <= Offset && Offset <= size()} && (Count
== dynamic_extent || Count >= 0 && Offset + Count <= size()).

-6- Effects: Equivalent to: return span<ElementType, see below>(
data() + Offset, Count != dynamic_extent ? Count : size() -
Offset);

-7- Remarks: The second template argument of the returned span type is:

Count != dynamic_extent ? Count
: (Extent != dynamic_extent ? Extent - Offset
: dynamic_extent)

constexpr span<element_type, dynamic_extent> first(index_type count) const;

-8- RequiresExpects: 0 <= count && count <= size().

-9- Effects: Equivalent to: return {data(), count};

constexpr span<element_type, dynamic_extent> last(index_type count) const;

-10- RequiresExpects: 0 <= count && count <= size().

-11- Effects: Equivalent to: return {data() + (size() - count),
count};

constexpr span<element_type, dynamic_extent> subspan(
index_type offset, index_type count = dynamic_extent) const;

-12- RequiresExpects: (0 <= offset && offset <= size()) &&
(count == dynamic_extent || count >= 0 && offset + count <=
size())

-13- Effects: Equivalent to: return {data() + offset, count ==
dynamic_extent ? size() - offset : count};

```

[2019-06-23; Tomasz comments and provides updated wording]

The current proposed resolution no longer applies to the newest revision of the standard ([N4820](#)), due changes introduced in [P1227](#) (making `size()` and template parameters of `span` unsigned).

[2019 Cologne Wednesday night]

Status to Ready

Proposed resolution:

This wording is relative to [N4820](#).

[Drafting note: This wording relies on observation, that the condition in form `Extent == dynamic_extent || Count <= Extent`, can be simplified into `Count <= Extent`, because `dynamic_extent` is equal to `numeric_limits<size_t>::max()`, thus `size() <= Extent` is always true, and `Extent == dynamic_extent` implies that `Count <= Extent`.

Furthermore we check that `Count != dynamic_extent || Count <= Extent - Offset`, as the `Offset + Count <= Extent` may overflow (defined for unsigned integers) and produce false positive result. This change is also applied to *Expects* clause.]

1. Edit 22.7.3.3 [\[span.sub\]](#) as indicated:

```
template<size_t Count> constexpr span<element_type, Count> first() const;
```

-?- Mandates: Count <= Extent is true.

-1- *Expects*: `Count <= size()` is true.

-2- *Effects*: Equivalent to: `return {data(), Count};`

```
template<size_t Count> constexpr span<element_type, Count> last() const;
```

-?- Mandates: Count <= Extent is true.

-3- *Expects*: `Count <= size()` is true.

-4- *Effects*: Equivalent to: `return {data() + (size() - Count), Count};`

```
template<size_t Offset, size_t Count = dynamic_extent>
constexpr span<element_type, see below> subspan() const;
```

-?- Mandates: Offset <= Extent && (Count == dynamic_extent || Count <= Extent - Offset) is true.

-5- *Expects*: `Offset <= size() && (Count == dynamic_extent || Offset + Count <= size() Count <= size() - Offset)` is true.

-6- *Effects*: Equivalent to: `return span<ElementType, see below>(data() + Offset, Count != dynamic_extent ? Count : size() - Offset);`

-7- *Remarks*: The second template argument of the returned span type is:

```
Count != dynamic_extent ? Count
                        : (Extent != dynamic_extent ? Extent - Offset
                                                : dynamic_extent)
```

[...]

```
constexpr span<element_type, dynamic_extent> subspan(
    index_type offset, index_type count = dynamic_extent) const;
```

-12- *Expects*: `offset <= size() && (count == dynamic_extent || offset + count <= size() count <= size() - offset)` is true.

-13- *Effects*: Equivalent to: `return {data() + offset, count == dynamic_extent ? size() - offset : count};`

[3149^{\(i\)}](#). DefaultConstructible should require default initialization

Section: 18.4.12 [\[concept.defaultconstructible\]](#) **Status:** [Ready](#) **Submitter:** Casey Carter **Opened:** 2018-08-09 **Last modified:** 2019-07-22

Priority: 2

View all issues with [Ready](#) status.

Discussion:

`DefaultConstructible<T>` is equivalent to `Constructible<T>` (18.4.11 [\[concept.constructible\]](#)), which is equivalent to `is_constructible_v<T>` (20.15.4.3 [\[meta.unary.prop\]](#)). Per 20.15.4.3 [\[meta.unary.prop\]](#) paragraph 8:

The predicate condition for a template specialization `is_constructible<T, Args...>` shall be satisfied if and only if the following variable definition would be well-formed for some invented variable `t`:

```
T t(declval<Args>()...);
```

`DefaultConstructible<T>` requires that objects of type `T` can be [value-initialized](#), rather than [default-initialized](#) as intended.

The library needs a constraint that requires object types to be default-initializable: the "rangified" versions of the algorithms in 20.10.11.3 [\[uninitialized.construct.default\]](#) proposed in [P0896 "The One Ranges Proposal"](#), for example. Users will also want a mechanism to provide such a constraint, and they're likely to choose `DefaultConstructible` despite its subtle unsuitability.

There are two alternative solutions: (1) change `DefaultConstructible` to require *default-initialization*, (2) change `is_default_constructible_v` to require default-initialization and specify the concept in terms of the trait. (2) is probably too breaking a change to be feasible.

[2018-08-20 Priority set to 2 after reflector discussion]

Previous resolution [SUPERSEDED]:

1. Modify 18.4.12 [\[concept.defaultconstructible\]](#) as follows:

```
template<class T>
    concept DefaultConstructible = Constructible<T> && see below;
```

!- Type `T` models `DefaultConstructible` only if the variable definition

`T t`;

is well-formed for some invented variable `t`. Access checking is performed as if in a context unrelated to `T`. Only the validity of the immediate context of the variable initialization is considered.

[2018-08-23 Tim provides updated P/R based on Batavia discussion]

[2018-10-28 Casey expands the problem statement and the P/R]

During Batavia review of [P0896R3](#), Tim Song noted that `{}` is not necessarily a valid initializer for a `DefaultConstructible` type. In this sample program (see [Compiler Explorer](#)):

```
struct S0 { explicit S0() = default; };
struct S1 { S0 x; }; // Note: aggregate
S1 x; // Ok
S1 y{}; // ill-formed; copy-list-initializes x from {}
```

`S1` can be default-initialized, but not list-initialized from an empty *braced-init-list*. The consensus among those present was that `DefaultConstructible` should prohibit this class of pathological types by requiring that initialization form to be valid.

[2019 Cologne Wednesday night]

Status to Ready

Proposed resolution:

This wording is relative to [N4762](#).

1. Modify 18.4.12 [\[concept.defaultconstructible\]](#) as follows:

```
template<class T>
    inline constexpr bool is-default-initializable = see below; // exposition only
```

```
template<class T>
    concept DefaultConstructible = Constructible<T> && requires { T{}; } && is-default-initializable<T>;
```

!- For a type `T`, `is-default-initializable<T>` is true if and only if the variable definition

`T t`;

is well-formed for some invented variable `t`; otherwise it is false. Access checking is performed as if in a context unrelated to `T`. Only the validity of the immediate context of the variable initialization is considered.

[3190](#)⁽ⁱ⁾. `std::allocator::allocate` sometimes returns too little storage

Section: 20.10.10.1 [\[allocator.members\]](#) **Status:** [Ready](#) **Submitter:** Casey Carter **Opened:** 2019-02-20 **Last modified:** 2019-07-22

Priority: 3

View other [active issues](#) in [\[allocator.members\]](#).

View all other [issues](#) in [\[allocator.members\]](#).

View all issues with [Ready](#) status.

Discussion:

20.10.10.1 [\[allocator.members\]](#)/2 says:

-2- *Returns:* A pointer to the initial element of an array of storage of size `n * sizeof(T)`, aligned appropriately for objects of type `T`.

As in LWG [3038](#), we should not return too little storage for `n` objects of size `sizeof(T)`, e.g. when `n` is `SIZE_MAX / 2` and `T` is short.

[2019-03-05 Priority set to 3 after reflector discussion]

[2019 Cologne Wednesday night]

Status to Ready; will open additional issue to reconcile this and [3038](#)

Proposed resolution:

This wording is relative to [N4800](#).

1. Change 20.10.10.1 [\[allocator.members\]](#) as indicated:

```
[[nodiscard]] T* allocate(size_t n);
```

```
...
```

-4- *Throws:* `bad_array_new_length` if `SIZE_MAX / sizeof(T) < n`, or `bad_alloc` if the storage cannot be obtained.

[3218](#)⁽ⁱ⁾. Modifier for `%d` parse flag does not match POSIX and format specification

Section: 27.13 [\[time.parse\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-06-13 **Last modified:** 2019-09-25

Priority: 0

View other [active issues](#) in [\[time.parse\]](#).

View all other [issues](#) in [\[time.parse\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

Currently, the `%d` parse flags accepts the `'E'` modifier to parse the locale's alternative representation, see Table 88 — ["Meaning of parse flags"](#):

The modified command `%Ed` interprets the locale's alternative representation of the day of the month.

This is inconsistent with the [POSIX strftime specification](#) and the `format` functions, that uses 'o' to output alternate locale representation. Per Table 87 — ["Meaning of format conversion specifiers"](#):

The modified command `%od` produces the locale's alternative representation.

[2019-06-24; Howard comments]

This was simply a type-o in my documentation that infected the proposal and subsequently the C++ working draft.

None of `std::time_put`, C's `strftime`, or POSIX's `strftime` support `%Ed` but all support `%od`. Furthermore the existing example implementation supports `%od` but not `%Ed`. And all the existing example implementation does is forward to `std::time_put`.

[2019-07 Issue Prioritization]

Status to Tentatively Ready after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4810](#).

1. Modify 27.13 [\[time.parse\]](#), Table 88 — "Meaning of parse flags", as indicated:

Table 88 — Meaning of parse flags	
Flag	Parsed value
	[...]
%d	The day of the month as a decimal number. The modified command <code>%Nd</code> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command <code>%Ed</code> interprets the locale's alternative representation of the day of the month.
	[...]

[3221\(i\)](#). Result of `year_month` arithmetic with months is ambiguous

Section: 27.8.13.3 [\[time.cal.ym.nonmembers\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-06-16 **Last modified:** 2019-09-25

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

The current specification of the addition of `year_month` and months does not define a unique result value.

To illustrate, both `year(2019)/month(1)` and `year(2018)/month(13)` are valid results of `year(2018)/month(12) + months(1)` addition, according to the spec in 27.8.13.3 [\[time.cal.ym.nonmembers\]](#).

[2019-06-24; LWG discussion]

During discussions on the LWG reflector there was a preference to add "is true" at the end of the modified *Returns:* element. This additional edit has been applied to Tomasz' original wording below.

[2019-07 Issue Prioritization]

Status to Tentatively Ready after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4810](#).

1. Modify 27.8.13.3 [\[time.cal.ym.nonmembers\]](#) as indicated:

```
constexpr year_month operator+(const year_month& ym, const months& dm) noexcept;

-3- Returns: A year_month value z such that z.ok() && z - ym == dm is true.
```

[3222\(i\)](#). P0574R1 introduced preconditions on non-existent parameters

Section: 25.9.8 [\[inclusive.scan\]](#), 25.9.10 [\[transform.inclusive.scan\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2019-06-18 **Last modified:** 2019-09-25

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

after applying [P0574R1](#) to the working draft, 25.9.10 [\[transform.inclusive.scan\]](#) bullet 1.1 says "If `init` is provided, [...]; otherwise, `ForwardIterator1`'s value type shall be [...]." 25.9.10 [\[transform.inclusive.scan\]](#) bullet 1.2 says "If `init` is provided, [...]; otherwise, [...] shall be convertible to `ForwardIterator1`'s value type."

For the first overload `init` is not provided, but there is no `ForwardIterator1`, so these requirements cannot be met. The requirements for the first overload need to be stated in terms of `InputIterator`'s value type, not `ForwardIterator1`'s value type.

The same problem exists in 25.9.8 [\[inclusive.scan\]](#). 25.9.11 [\[adjacent.difference\]](#) solves this problem by saying "Let τ be the value type of `decltype(first)`".

[2019-07 Issue Prioritization]

Status to Tentatively Ready after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4820](#).

1. Modify 25.9.8 [\[inclusive.scan\]](#) as indicated:

```
template<class InputIterator, class OutputIterator, class BinaryOperation>
    OutputIterator inclusive_scan(InputIterator first, InputIterator last,
                                OutputIterator result, BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation>
    ForwardIterator2 inclusive_scan(ExecutionPolicy&& exec,
                                ForwardIterator1 first, ForwardIterator1 last,
                                ForwardIterator2 result, BinaryOperation binary_op);
template<class InputIterator, class OutputIterator, class BinaryOperation, class T>
    OutputIterator inclusive_scan(InputIterator first, InputIterator last,
                                OutputIterator result, BinaryOperation binary_op, T init);
template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2, class BinaryOperation, class T>
    ForwardIterator2 inclusive_scan(ExecutionPolicy&& exec,
                                ForwardIterator1 first, ForwardIterator1 last,
                                ForwardIterator2 result, BinaryOperation binary_op, T init);
```

-?- Let U be the value type of `decltype(first)`.

-3- Requires:

(3.1) — If `init` is provided, τ shall be *Cpp17MoveConstructible* (Table 26); otherwise, ~~`ForwardIterator1`'s value type~~ U shall be *Cpp17MoveConstructible*.

(3.2) — If `init` is provided, all of `binary_op(init, init)`, `binary_op(init, *first)`, and `binary_op(*first, *first)` shall be convertible to T ; otherwise, `binary_op(*first, *first)` shall be convertible to ~~`ForwardIterator1`'s value type~~ U .

(3.3) — `binary_op` shall neither invalidate iterators or subranges, nor modify elements in the ranges `[first, last]` or `[result, result + (last - first)]`.

2. Modify 25.9.10 [\[transform.inclusive.scan\]](#) as indicated:

```
template<class InputIterator, class OutputIterator,
        class BinaryOperation, class UnaryOperation>
```

```

        OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                              OutputIterator result,
                                              BinaryOperation binary_op, UnaryOperation unary_op);
template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation, class UnaryOperation>
ForwardIterator2 transform_inclusive_scan(ExecutionPolicy&& exec,
                                        ForwardIterator1 first, ForwardIterator1 last,
                                        ForwardIterator2 result,
                                        BinaryOperation binary_op, UnaryOperation unary_op);
template<class InputIterator, class OutputIterator,
        class BinaryOperation, class UnaryOperation, class T>
OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                        OutputIterator result,
                                        BinaryOperation binary_op, UnaryOperation unary_op,
                                        T init);
template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation, class UnaryOperation, class T>
ForwardIterator2 transform_inclusive_scan(ExecutionPolicy&& exec,
                                        ForwardIterator1 first, ForwardIterator1 last,
                                        ForwardIterator2 result,
                                        BinaryOperation binary_op, UnaryOperation unary_op,
                                        T init);

```

-?- Let U be the value type of `decltype(first)`.

-1- Requires:

(1.1) — If `init` is provided, `T` shall be *Cpp17MoveConstructible* (Table 26); otherwise, ~~ForwardIterator1's value type~~ **U** shall be *Cpp17MoveConstructible*.

(1.2) — If `init` is provided, all of

(1.2.1) — `binary_op(init, init)`,

(1.2.2) — `binary_op(init, unary_op(*first))`, and

(1.2.3) — `binary_op(unary_op(*first), unary_op(*first))`

shall be convertible to `T`; otherwise, `binary_op(unary_op(*first), unary_op(*first))` shall be convertible to ~~ForwardIterator1's value type~~ **U**.

(1.3) — Neither `unary_op` nor `binary_op` shall invalidate iterators or subranges, nor modify elements in the ranges `[first, last]` or `[result, result + (last - first)]`.

[3224^{\(i\)}](#). `zoned_time` constructor from `TimeZonePtr` does not specify initialization of `tp_`

Section: 27.11.7.2 [[time.zone.zonedtime.ctor](#)] **Status:** [Tentatively Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-06-20 **Last modified:** 2019-09-25

Priority: 0

View other [active issues](#) in [`time.zone.zonedtime.ctor`].

View all other [issues](#) in [`time.zone.zonedtime.ctor`].

View all issues with [Tentatively Ready](#) status.

Discussion:

The `zoned_time(TimeZonePtr z)` does not specify how the `tp_` sub-element is initialized. It should be consistent with `zoned_time(string_view)` that default initializes `tp_`.

[2019-07 Issue Prioritization]

Status to Tentatively Ready after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4820](#).

1. Modify 27.11.7.2 [\[time.zone.zonedtime.ctor\]](#) as indicated:

```
explicit zoned_time(TimeZonePtr z);
```

-5- *Requires*: z refers to a time zone.

-6- *Effects*: Constructs a zoned_time by initializing zone_ with std::move(z) [and default constructing tp_](#).

[3225^{\(i\)}](#). zoned_time converting constructor shall not be noexcept

Section: 27.11.7.2 [\[time.zone.zonedtime.ctor\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-06-20 **Last modified:** 2019-09-25

Priority: 0

View other [active issues](#) in [\[time.zone.zonedtime.ctor\]](#).

View all other [issues](#) in [\[time.zone.zonedtime.ctor\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

The zoned_time constructor from zoned_time<Duration2, TimeZonePtr> (preserving same time zone, different precision of representation) is currently marked noexcept. Given that the *exposition-only* member tp_ of type sys_time<duration> has essentially type time_point<system_clock, Duration>, this is incompatible with the invoked time_point constructor, which is not marked as noexcept.

[2019-07 Issue Prioritization]

Status to Tentatively Ready after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4820](#).

1. Modify 27.11.7.1 [\[time.zone.zonedtime.overview\]](#), class template zoned_time synopsis, as indicated:

```
template<class Duration2>
    zoned_time(const zoned_time<Duration2, TimeZonePtr>& zt) noexcept;
```

2. Modify 27.11.7.2 [\[time.zone.zonedtime.ctor\]](#) as indicated:

```
template<class Duration2>
    zoned_time(const zoned_time<Duration2, TimeZonePtr>& y) noexcept;
```

-9- *Requires*: Does not participate in overload resolution unless sys_time<Duration2> is implicitly convertible to sys_time<Duration>.

-10- *Effects*: Constructs a zoned_time by initializing zone_ with y.zone_ and tp_ with y.tp_.

[3230^{\(i\)}](#). Format specifier %y/%Y is missing locale alternative versions

Section: 27.13 [\[time.parse\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-06-29 **Last modified:** 2019-09-25

Priority: 0

View other [active issues](#) in [\[time.parse\]](#).

View all other [issues](#) in [\[time.parse\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

The year format specifier ('y', 'Y') are missing the locale alternative version ('%EY', '%Ey' and '%Oy'). That makes it inconsistent with the [POSIX strftime specification](#):

%Ey Replaced by the offset from %EC (year only) in the locale's alternative representation.

%EY Replaced by the full alternative year representation.

%Oy Replaced by the year (offset from %C) using the locale's alternative numeric symbols.

and parse specifiers 27.13 [\[time.parse\]](#) that accepts these modified command.

[2019-07 Issue Prioritization]

Status to Tentatively Ready after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4820](#).

[Drafting note: For the '%Oy' specifier we preserve consistency with the current specification for '%Od' and '%Oe' from Table 87 "Meaning of format conversion specifier" [tab:time.format.spec]:

%d [...] The modified command %Od produces the locale's alternative representation.

%e [...] The modified command %Oe produces the locale's alternative representation.

as their corresponding POSIX specification is matching one for '%Oy':

%Od Replaced by the day of the month, using the locale's alternative numeric symbols, filled as needed with leading zeros if there is any alternative symbol for zero; otherwise, with leading <space> characters.

%Oe Replaced by the day of the month, using the locale's alternative numeric symbols, filled as needed with leading <space> characters.

]

1. Modify "Table 87 — Meaning of format conversion specifiers" [tab:time.format.spec] as indicated:

Table 87 — Meaning of format conversion specifiers [tab:time.format.spec]

Specifier	Replacement
	[...]
%y	The last two decimal digits of the year. If the result is a single digit it is prefixed by 0. The modified command %Oy produces the locale's alternative representation. The modified command %Ey produces the locale's alternative representation of offset from %EC (year only).
%Y	The year as a decimal number. If the result is less than four digits it is left-padded with 0 to four digits. The modified command %EY produces the locale's alternative full year representation.
	[...]

[3231^{\(i\)}](#). year_month_day_last::day specification does not cover !ok() values

Section: 27.8.15.2 [\[time.cal.ymdlast.members\]](#) Status: [Tentatively Ready](#) Submitter: Tomasz Kamiński Opened: 2019-06-29 Last modified: 2019-09-25

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

The current specification of the `year_month_day_last::day` function does not cover the behaviour in the situation when `year_month_day_last` value is not `ok()`. To illustrate the sentence from 27.8.15.2 [\[time.cal.ymdlast.members\]](#) p13:

A day representing the last day of the (year, month) pair represented by `*this`.

is unclear in the situation when month member has `!ok` value, e.g. what is last day of 14th month of 2019.

The proposed resolution makes the value of `ymd1.day()` (and by consequence conversion to `sys_days/local_days`) unspecified if `ymd1.ok()` is false. This make is consistent with rest of the library, that produces unspecified values in similar situation, e.g.: 27.8.14.2 [\[time.cal.ymd.members\]](#) p18, 27.8.16.2 [\[time.cal.ymwd.members\]](#) p19, 27.8.17.2 [\[time.cal.ymwdlast.members\]](#) p14.

[2019-07 Issue Prioritization]

Status to Tentatively Ready after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4820](#).

1. Modify 27.8.15.2 [\[time.cal.ymdlast.members\]](#) as indicated:

```
constexpr chrono::day day() const noexcept;
```

-13- Returns: If `ok()` is true, returns a day representing the last day of the (year, month) pair represented by `*this`. Otherwise the returned value is unspecified.

-14- [Note: This value may be computed on demand. — end note]

[3232^{\(i\)}](#). Inconsistency in zoned_time deduction guides

Section: 27.11.7.1 [\[time.zone.zonedtime.overview\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-06-30
Last modified: 2019-09-25

Priority: 0

View other [active issues](#) in [\[time.zone.zonedtime.overview\]](#).

View all other [issues](#) in [\[time.zone.zonedtime.overview\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

Currently, the `time_zone` is always storing the time with the precision not coarser than seconds, as consequence any instance with the duration with coarser precision with seconds, is de-facto equivalent to one instantiated to the duration of seconds. This is caused by the fact, that time zone offset can be defined up to seconds precision. To illustrate both of following specialization has the same behavior (keep seconds):

```
zoned_time<minutes> zt(current_zone(), floor<minutes>(system_clock::now()));
zoned_time<hours>   zt(current_zone(), floor<hours>(system_clock::now()));
zoned_time<seconds> zt(current_zone(), floor<seconds>(system_clock::now()));
```

To avoid unnecessary code bloat caused by above, most deduction guides are instantiating `zoned_time` with at least seconds precision (i.e. `zoned_time<seconds>` will be deduced for all of above):

```
template<class TimeZonePtr, class Duration>
    zoned_time(TimeZonePtr, zoned_time<Duration>, choose = choose::earliest)
    -> zoned_time<common_type_t<Duration, seconds>, TimeZonePtr>;
```

However there is single exception:

```
template<class Duration, class TimeZonePtr, class TimeZonePtr2>
    zoned_time(TimeZonePtr, zoned_time<Duration, TimeZonePtr2>, choose = choose::earliest)
    -> zoned_time<Duration, TimeZonePtr>;
```

This deduction guide should be updated to preserve the consistency of design.

[2019-07 Issue Prioritization]

Status to Tentatively Ready after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4820](#).

1. Modify 27.11.7.1 [\[time.zone.zonedtime.overview\]](#) as indicated:

```
[...]
template<class Duration, class TimeZonePtr, class TimeZonePtr2>
    zoned_time(TimeZonePtr, zoned_time<Duration, TimeZonePtr2>, choose = choose::earliest)
    -> zoned_time<common_type_t<Duration, seconds>, TimeZonePtr>;
[...]
```

[3235\(i\)](#). parse manipulator without abbreviation is not callable

Section: 27.13 [\[time.parse\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-07-10 **Last modified:** 2019-09-25

Priority: 0

View other [active issues](#) in [time.parse].

View all other [issues](#) in [time.parse].

View all issues with [Tentatively Ready](#) status.

Discussion:

The parse overload that does not accept the abbreviation but does accept an offset, because the expression in the *Remarks*: clause:

```
from_stream(declval<basic_istream<charT, traits>*>(), fmt.c_str(), tp, nullptr, &offset)
```

is not valid. This is caused by deduction failure for the `basic_string<charT, traits, Alloc>*` from `nullptr` (see [this link](#)):

[2019-08-17 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after six positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

[Drafting note: As a drive-by fix the *Remarks* element is also converted to a *Constraints* element.]

1. Modify 27.13 [\[time.parse\]](#) as indicated:

```
template<class charT, class traits, class Alloc, class Parsable>
    unspecified
    parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
          minutes& offset);
```

-6- ~~Remarks~~ **Constraints:** ~~This function shall not participate in overload resolution unless~~ **The expression**

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str(), tp,
    declval<basic_string<charT, traits, Alloc>*>(), nullptr, &offset)
```

is a valid well-formed expression when treated as an unevaluated operand.

-7- *Returns:* A manipulator that, when extracted from a `basic_istream<charT, traits>` is, calls `from_stream(is, fmt.c_str(), tp, static_cast<basic_string<charT, traits, Alloc>*>(nullptr), &offset)`.

[3241^{\(i\)}](#). chrono-spec grammar ambiguity in §[time.format]

Section: 27.12 [\[time.format\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Victor Zverovich **Opened:** 2019-07-24 **Last modified:** 2019-09-25

Priority: 0

View other [active issues](#) in [time.format].

View all other [issues](#) in [time.format].

View all issues with [Tentatively Ready](#) status.

Discussion:

The chrono-spec grammar introduced by [P1361R2](#) "Integration of chrono with text formatting" in section [time.format] is ambiguous because '%' can be interpreted as either literal-char or conversion-spec:

```
chrono-spec      ::= literal-char | conversion-spec
literal-char     ::= <a character other than '{' or '}'>
conversion-spec  ::= '%' [modifier] type
```

[2019-08-17 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify the literal-char grammar in 27.12 [\[time.format\]](#) as indicated:

```
literal-char:
    any character other than {, or }, or %
```

[3244^{\(i\)}](#). Constraints for source in §[fs.path.req] insufficiently constrainty

Section: 29.11.7.3 [\[fs.path.req\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2019-08-02 **Last modified:** 2019-09-25

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

std::filesystem::path has a number of functions - notably including a conversion constructor template (29.11.7.4.1 [\[fs.path.construct\]](#)) and assignment operator template (29.11.7.4.2 [\[fs.path.assign\]](#)) - that accept const Source&. Per 29.11.7.3 [\[fs.path.req\]](#) paragraph 2:

-2- Functions taking template parameters named Source shall not participate in overload resolution unless either

(2.1) — Source is a specialization of basic_string or basic_string_view, or

(2.2) — the *qualified-id* iterator_traits<decay_t<Source>>::value_type is valid and denotes a possibly const encoded character type.

iterator_traits<decay_t<path>>::value_type is not valid in C++17, so this specification was sufficient to guard against the conversion constructor template (respectively assignment operator template) "pretending" to be copy constructor (respectively copy assignment operator). [P0896R4 "The One Ranges Proposal"](#), however, altered the definition of iterator_traits in the working draft. It now has some convenient default behaviors for types that meet (roughly) the syntax of the Cpp17InputIterator requirements. Notably those requirements include copy construction and copy assignment.

In the working draft, to determine the copyability of std::filesystem::path we must perform overload resolution to determine if we can initialize a path from a constant lvalue of type path. The conversion constructor template that accepts const Source& is a candidate, since its second argument is defaulted, so we must perform template argument deduction to see if this constructor is viable. Source is deduced to path and we then must check the constraint from 29.11.7.3 [\[fs.path.req\]](#) paragraph 2.2 (above). Checking the

constraint requires us to specialize `iterator_traits<path>`, which (per 23.3.2.3 [\[iterator.traits\]](#) paragraph 3.2) requires us to determine if `path` satisfies the exposition-only `cpp17-input-iterator` concept, which requires `path` to be copyable.

We've completed a cycle: determining if `path` is copyable requires us to first determine if `path` is copyable. This unfortunate constraint recursion can be broken by explicitly specifying that `path` is not a valid `Source`.

[2019-08-17 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after seven positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

- 1. Modify 29.11.7.3 [\[fs.path.req\]](#) as indicated:
 - 2- Functions taking template parameters named `Source` shall not participate in overload resolution unless `Source` denotes a type other than `path`, and either
- [...]

[3245\(i\)](#). Unnecessary restriction on '%p' parse specifier

Section: 27.13 [\[time.parse\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-07-31 **Last modified:** 2019-09-25

Priority: 0

View other [active issues](#) in `[time.parse]`.

View all other [issues](#) in `[time.parse]`.

View all issues with [Tentatively Ready](#) status.

Discussion:

The current specification for the '%p' flag in "[tab:time.parse.spec] Meaning of parse flags" places a restriction of it's placement with regards to the '%I' command:

The locale's equivalent of the AM/PM designations associated with a 12-hour clock. The command %I must precede %p in the format string.

This restriction makes the migration to new API more difficult, as it is not present for the [POSIX strptime](#) nor in the [example implementation](#) of the library. Per Howard's comment:

Actually this is an obsolete requirement and it should be struck. The first time I implemented this I didn't know how to do it without this requirement. I've since reimplemented it without needing this.

[2019-08-17 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after eight positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

- 1. Modify Table 99 "Meaning of parse flags [tab:time.parse.spec]" in 27.13 [\[time.parse\]](#) as indicated:

Table 99: Meaning of parse flags [tab:time.parse.spec]	
Flag	Parsed value
	[...]
%p	The locale's equivalent of the AM/PM designations associated with a 12-hour clock. The command %I must precede %p in the format string.
	[...]

[3246^{\(i\)}](#). What are the constraints on the template parameter of `basic_format_arg`?

Section: 20.20.5.1 [\[format.arg\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2019-08-01 **Last modified:** 2019-09-25

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

In [P0645R10](#) 20.?.5.1/ we find:

Constraints: `typename Context::template formatter_type<T>` is enabled.

... which doesn't mean anything, because that is an arbitrary type. Presumably the intent is that that type will always be a specialization of `formatter`, but there appear to be no constraints whatsoever on the `Context` template parameter, so there seems to be no requirement that that is the case.

Should `basic_format_arg` place some constraints on its `Context` template parameter? E.g., should it be required to be a specialization of `basic_format_context`?

Victor Zverovich:

The intent here is to allow different context types provide their own formatter extension types. The default formatter context and extension are `basic_format_context` and `formatter` respectively, but it's possible to have other. For example, in the `fmt` library there is a formatter context that supports `printf` formatting for legacy code. It cannot use the default formatter specializations because of the different syntax (`%...` vs `{...}`).

Richard Smith:

In either case, the specification here seems to be missing the rules for what is a valid `Context` parameter.

I'm happy to editorially change "is enabled" to "is an enabled specialization of `formatter`", since there's nothing else that this could mean, but we still need a wording fix for the broader issue here. Here's what I have so far for this wording:

Constraints: The template specialization `typename Context::template formatter_type<T>` is an enabled specialization of `formatter` (`[formatter.requirements]`).

Tim Song:

I think what we actually want here is "`typename Context::template formatter_type<T>` meets the *Formatter* requirements".

[2019-08-17 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after six positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 20.20.5.1 [\[format.arg\]](#) as indicated:

```
template<class T> explicit basic_format_arg(const T& v) noexcept;
```

-4- *Constraints:* The template specialization

```
typename Context::template formatter_type<T>
```

~~is an enabled specialization of `formatter`~~ **meets the *Formatter* requirements** (20.20.4 [\[format.formatter\]](#)). The extent to which an implementation determines that the specialization ~~is enabled~~ **meets the *Formatter* requirements** is unspecified, except that as a minimum the expression

```
typename Context::template formatter_type<T>()  
.format(declval<const T&>(), declval<Context&>())
```

shall be well-formed when treated as an unevaluated operand.

[3253^{\(i\)}](#). `basic_syncbuf::basic_syncbuf()` should not be explicit

Section: 29.10.2.1 [[syncstream.syncbuf.overview](#)] **Status:** [Tentatively Ready](#) **Submitter:** Nevin Liber **Opened:** 2019-08-06 **Last modified:** 2019-09-25

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

When [P0935](#) "Eradicating unnecessarily explicit default constructors from the standard library" was applied, `basic_syncbuf` was not in the working paper. `basic_syncbuf` should not have an explicit default constructor.

[2019-09-02 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after seven positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 29.10.2.1 [[syncstream.syncbuf.overview](#)], class template `basic_syncbuf` synopsis, as indicated:

```
template<class charT, class traits, class Allocator>
class basic_syncbuf : public basic_streambuf<charT, traits> {
public:
    [...]
    // 29.10.2.2 [syncstream.syncbuf.cons], construction and destruction
    basic_syncbuf()
    : basic_syncbuf(nullptr) {}
    explicit basic_syncbuf(streambuf_type* obuf = nullptr)
    : basic_syncbuf(obuf, Allocator()) {}
    [...]
};
```

[3256^{\(i\)}](#). Feature testing macro for constexpr algorithms

Section: 17.3.1 [[support.limits.general](#)] **Status:** [Tentatively Ready](#) **Submitter:** Antony Polukhin **Opened:** 2019-08-14 **Last modified:** 2019-09-25

Priority: 0

View other [active issues](#) in [[support.limits.general](#)].

View all other [issues](#) in [[support.limits.general](#)].

View all issues with [Tentatively Ready](#) status.

Discussion:

Feature testing macro from [P0202](#) "Add Constexpr Modifiers to Functions in <algorithm> and <utility> Headers" is missing in the WD.

For user convenience and to reduce feature testing macro count it would be better to stick to initial version of P0202 that was providing only the `__cpp_lib_constexpr_algorithms`.

So remove `__cpp_lib_constexpr_swap_algorithms`, define `__cpp_lib_constexpr_algorithms` to 201703L if [P0202](#) is implemented, to 201806L if [P0202](#)+[P0879](#) are implemented.

[2019-09-02 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify the Table 36 "Standard library feature-test macros" [tab:support.ft] in 17.3.1 [\[support.limits.general\]](#) as indicated:

Table 36: Standard library feature-test macros [tab:support.ft]

Macro name	Value	Header(s)
[...]		
__cpp_lib_constexpr_swap_algorithms	201806L	<algorithm>
[...]		

[3257](#)⁽ⁱ⁾. Missing feature testing macro update from P0858

Section: 17.3.1 [\[support.limits.general\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Antony Polukhin **Opened:** 2019-08-14 **Last modified:** 2019-09-25

Priority: 0

View other [active issues](#) in [support.limits.general].

View all other [issues](#) in [support.limits.general].

View all issues with [Tentatively Ready](#) status.

Discussion:

[P0858](#) "Constexpr iterator requirements" suggested to update the feature-testing macros `__cpp_lib_string_view` and `__cpp_lib_array_constexpr` to the date of adoption.

That did not happen.

[2019-09-02 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify the Table 36 "Standard library feature-test macros" [tab:support.ft] in 17.3.1 [\[support.limits.general\]](#) as indicated:

Table 36: Standard library feature-test macros [tab:support.ft]

Macro name	Value	Header(s)
[...]		
__cpp_lib_array_constexpr	201803L	<iterator> <array>
[...]		
__cpp_lib_string_view	201603L	<string> <string_view>
[...]		

[3259](#)⁽ⁱ⁾. The definition of *constexpr iterators* should be adjusted

Section: 23.3.1 [\[iterator.requirements.general\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Daniel Krüger **Opened:** 2019-08-18 **Last modified:** 2019-09-25

Priority: 0

View other [active issues](#) in [iterator.requirements.general].

View all other [issues](#) in [iterator.requirements.general].

View all issues with [Tentatively Ready](#) status.

Discussion:

The current definition of *constexpr iterators* is specified in 23.3.1 [\[iterator.requirements.general\]](#) p16 as follows:

Iterators are called *constexpr iterators* if all operations provided to meet iterator category requirements are *constexpr* functions, except for

(16.1) — a pseudo-destructor call (7.5.4.3 [\[expr.prim.id.dtor\]](#)), and

(16.2) — the construction of an iterator with a singular value.

With the acceptance of some proposals during the Cologne 2019 meeting, these additional requirements become mostly obsolete, as it had already been pointed out during that meeting:

With the acceptance of [P0784R7](#), destructors can be declared *constexpr* and it is possible to perform a pseudo-destructor call within a constant expression, so bullet (16.1) is no longer a necessary requirement.

With the acceptance of [P1331R2](#), trivial default initialization in *constexpr* contexts is now possible, and there is no longer a requirement to initialize all sub-objects of a class object within a constant expression.

It seems to me that we should simply strike the above two constraining requirements of the definition of *constexpr iterators* for C++20.

[2019-09-14 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 23.3.1 [\[iterator.requirements.general\]](#) as indicated:

-16- Iterators are called *constexpr iterators* if all operations provided to meet iterator category requirements are *constexpr* functions, ~~except for~~

~~(16.1) — a pseudo-destructor call (7.5.4.3 [\[expr.prim.id.dtor\]](#)), and~~

~~(16.2) — the construction of an iterator with a singular value.~~

[3266^{\(i\)}](#). `to_chars(bool)` should be deleted

Section: 20.19.1 [\[charconv.syn\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jens Maurer **Opened:** 2019-08-23 **Last modified:** 2019-09-25

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

20.19.2 [\[charconv.to.chars\]](#) does not present an overload for `bool` (because it is neither a signed nor unsigned integer type), so an attempt to call `to_chars` with a `bool` argument would promote it to `int` and unambiguously call the `int` overload of `to_chars`.

This was not intended, since it is not obvious that the correct textual representation of a `bool` is `0/1` (as opposed to, say, `"true"/"false"`).

The user should cast explicitly if he wants the `0/1` behavior. (Correspondingly, there is no `bool` overload for `from_chars` in the status quo, and conversions do not apply there because of the reference parameter.)

[2019-09-14 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after eight positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 20.19.1 [\[charconv.syn\]](#), header `<charconv>` synopsis, as indicated:

```
[...]
// 20.19.2 \[charconv.to.chars\], primitive numerical output conversion
```

```

struct to_chars_result {
    char* ptr;
    errc ec;
    friend bool operator==(const to_chars_result&, const to_chars_result&) = default;
};

to_chars_result to_chars(char* first, char* last, see below value, int base = 10);
to_chars_result to_chars(char* first, char* last, bool value, int base = 10) = delete;
to_chars_result to_chars(char* first, char* last, float value);
[...]
```

3272(i). %I%p should parse/format duration since midnight

Section: 27.12 [\[time.format\]](#), 27.13 [\[time.parse\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Howard Hinnant **Opened:** 2019-09-02 **Last modified:** 2019-10-06

Priority: 0

View other [active issues](#) in [time.format].

View all other [issues](#) in [time.format].

View all issues with [Tentatively Ready](#) status.

Discussion:

It is clear how "%I%p" parses and formats time points. What is not clear is how these flags interact with durations. We should treat durations as "elapsed time since midnight". For example:

```

#include <chrono>
#include <iostream>
#include <string>
#include <sstream>

int main()
{
    using namespace std;
    using namespace std::chrono;
    // Format
    {
        // format time_point with %I%p
        cout << format("{:%F %I%p}", sys_days{2019_y/August/10}+14h) << '\n';
    }
    {
        // format duration with %I%p
        cout << format("{:%I%p}", 14h) << '\n';
    }

    // Parse
    {
        // Parse %I%p as day-of-year combined with an hour into a time_point
        istringstream in{"2019-08-10 2pm"};
        system_clock::time_point tp;
        in >> parse("%F %I%p", tp);
        cout << tp << '\n';
    }
    {
        // Parse %I%p as number of hours into a duration
        istringstream in{"2pm"};
        hours d;
        in >> parse("%I%p", d);
        cout << d << '\n';
    }
}
```

Output:

```

2019-08-10 02PM
02PM
2019-08-10 14:00:00.000000
14h
```

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 27.12 [\[time.format\]](#) as indicated:

-3- Unless explicitly requested, the result of formatting a chrono type does not contain time zone abbreviation and time zone offset information. If the information is available, the conversion specifiers %Z and %z will format this information (respectively). [Note: If the information is not available and a %Z or %z conversion specifier appears in the *chrono-format-spec*, an exception of type `format_error` is thrown, as described above. — end note]

-?- If the type being formatted does not contain the information that the format flag needs, an exception of type `format_error` is thrown. [Example: A duration does not contain enough information to format as a weekday — end example]. However if a flag refers to a "time of day" (e.g. %H, %I, %p, etc.), then a specialization of duration is interpreted as the time of day elapsed since midnight.

2. Modify 27.13 [\[time.parse\]](#) as indicated:

[Drafting note: The modification of 27.13 [\[time.parse\]](#) p1 is intended to be non-conflictingly mergeable with the change suggested by LWG [3269](#) and LWG [3271](#) at the same paragraph.]

-1- Each parse overload specified in this subclause calls `from_stream` unqualified, so as to enable argument dependent lookup (6.5.2 [\[basic.lookup.argdep\]](#)). In the following paragraphs, let `is` denote an object of type `basic_istream<charT, traits>`, where `charT` and `traits` are template parameters in that context.

[...]

-10- All `from_stream` overloads behave as unformatted input functions, except that they have an unspecified effect on the value returned by subsequent calls to `basic_istream<>::gcount()`. Each overload takes a format string containing ordinary characters and flags which have special meaning. Each flag begins with a %. Some flags can be modified by E or O. During parsing each flag interprets characters as parts of date and time types according to Table [\[tab:time.parse.spec\]](#). Some flags can be modified by a width parameter given as a positive decimal integer called out as *N* below which governs how many characters are parsed from the stream in interpreting the flag. All characters in the format string that are not represented in Table [\[tab:time.parse.spec\]](#), except for white space, are parsed unchanged from the stream. A white space character matches zero or more white space characters in the input stream.

-?- If the type being parsed can not represent the information that the format flag refers to, `is.setstate(ios_base::failbit)` is called. [Example: A duration cannot represent a weekday — end example]. However if a flag refers to a "time of day" (e.g. %H, %I, %p, etc.), then a specialization of duration is parsed as the time of day elapsed since midnight.

[3273](#)⁽ⁱ⁾. Specify `weekday_indexed` to range of [0, 7]

Section: 27.8.7.2 [\[time.cal.wdidx.members\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Howard Hinnant **Opened:** 2019-09-02 **Last modified:** 2019-09-25

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

On one hand, we say that if you try to construct a `weekday_indexed` with index 0, you get an unspecified index instead (27.8.7.2 [\[time.cal.wdidx.members\]](#)/p1):

The values held are unspecified if `!wd.ok()` or index is not in the range [1, 5].

OTOH, we take pains to pin down `year_month_weekday`'s conversion to `sys_days` when the index is zero (27.8.7.2 [\[time.cal.wdidx.members\]](#)/p19):

If `index()` is 0 the returned `sys_days` represents the date 7 days prior to the first `weekday()` of `year()/month()`.

This is inconsistent. We should allow a slightly wider range (say, `[0, 7]`, since you need at least 3 bits anyway to represent the 5 distinct valid values, and the 0 value referred to by 27.8.7.2 [\[time.cal.wdidx.members\]](#)/p19.

[2019-09-24 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after six positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 27.8.7.2 [\[time.cal.wdidx.members\]](#) as indicated:

[Drafting note: As a drive-by fix a cleanup of "Constructs an object of type `weekday_indexed`" wording has been applied as well.]

```
constexpr weekday_indexed(const chrono::weekday& wd, unsigned index) noexcept;
```

-1- Effects: ~~Constructs an object of type `weekday_indexed` by initializing~~`Initializes` `wd_` with `wd` and `index_` with `index`. The values held are unspecified if `!wd.ok()` or `index` is not in the range `[10, 57]`.

[3274](#)⁽ⁱ⁾. Missing feature test macro for `<span>`

Section: 17.3.1 [\[support.limits.general\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2019-09-05 **Last modified:** 2019-09-25

Priority: 0

View other [active issues](#) in `[support.limits.general]`.

View all other [issues](#) in `[support.limits.general]`.

View all issues with [Tentatively Ready](#) status.

Discussion:

There is no feature test macro for `std::span`.

For the purposes of SD-6, I think we want two values: 201803L for the original addition of `<span>` by [P0122R7](#) (Jacksonville, 2018) and then 201902L for the API changes from [P1024R3](#) (Kona, 2019). The C++ working draft only needs the newer value.

[2019-09-24 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after eight positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify the Table 36 "Standard library feature-test macros" `[tab:support.ft]` in 17.3.1 [\[support.limits.general\]](#) as indicated:

Table 36: Standard library feature-test macros `[tab:support.ft]`

Macro name	Value	Header(s)
[...]		
<code>cpp_lib_span</code>	201902L	<code></code>
[...]		

[3276](#)⁽ⁱ⁾. Class `split_view::outer_iterator::value_type` should inherit from `view_interface`

Section: 24.7.11.4 [\[range.split.outer.value\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Eric Niebler **Opened:** 2019-09-09 **Last modified:** 2019-09-25

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

It is a view. It should have all the view goodies. Suggested priority P1 because it affects ABI.

The proposed change has been implemented and tested in range-v3.

[2019-09-24 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after six positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 24.7.11.4 [\[range.split.outer.value\]](#), class `split_view::outer_iterator::value_type` synopsis, as indicated:

```
namespace std::ranges {
    template<class V, class Pattern>
    template<bool Const>
    struct split_view<V, Pattern>::outer_iterator<Const>::value_type
        : view_interface<value_type> {
    private:
        outer_iterator i_ = outer_iterator(); // exposition only
    public:
        value_type() = default;
        constexpr explicit value_type(outer_iterator i);

        constexpr inner_iterator<Const> begin() const;
        constexpr default_sentinel_t end() const;
    };
}
```

[3277^{\(i\)}](#). Pre-increment on prvalues is not a requirement of weakly_incrementable

Section: 23.3.4.13 [\[iterator.concept.random.access\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Eric Niebler **Opened:** 2019-09-09 **Last modified:** 2019-09-25

Priority: 0

View other [active issues](#) in [\[iterator.concept.random.access\]](#).

View all other [issues](#) in [\[iterator.concept.random.access\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

See 23.3.4.13 [\[iterator.concept.random.access\]](#)/2.6, which shows ++ being applied to a prvalue iterator.

A similar change has already been made to 24.6.3.2 [\[range.iota.view\]](#)/4.6.

Suggest priority P0 or P1 because it effects the definition of a concept.

[2019-09-24 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after six positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 23.3.4.13 [\[iterator.concept.random.access\]](#) as indicated:

-2- Let a and b be valid iterators of type I such that b is reachable from a after n applications of $++a$, let D be `iter_difference_t<I>`, and let n denote a value of type D . I models `random_access_iterator` only if

(2.1) — $(a += n)$ is equal to b .

[...]

(2.6) — If $(a + D(n - 1))$ is valid, then $(a + n)$ is equal to `++[I c]{ return ++c; }(a + D(n - 1))`.

[...]